.UBAfiuba
FACULTAD DE INGENIERÍA

Clasificación automática de modulaciones de señales de radiofrecuencia mediante machine learning

Ing. Manuel Dieguez

Carrera de Especialización en Inteligencia Artificial

Director: Dr. Rodríguez Colmeiro, Ramiro Germán (UTN - UTT)

Codirector: Ing. De Cesare, Pablo Nicolás (UTN)

Jurados:

Jurado 1 (pertenencia)
Jurado 2 (pertenencia)
Jurado 3 (pertenencia)

Ciudad Autónoma de Buenos Aires, noviembre de 2025

Resumen

Se implementaron y compararon múltiples arquitecturas para la clasificación automática de modulaciones en señales de radio, con el fin de identificar la que ofreciera la mejor relación entre exactitud y complejidad computacional. La misma fue realizada en el marco de un emprendimiento personal para la Carrera de Especialización en Inteligencia Artificial, de la Facultad de Ingeniería de la Universidad de Buenos Aires (FIUBA). La memoria integra conocimientos de aprendizaje profundo, modelos clásicos y procesamiento de datos adquiridos en dicha carrera, y también conceptos de procesamiento de señales debido al transfondo del autor en la rama de telecomunicaciones.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen I

1. Introducción general 1
1.1. Reconocimiento automático de modulaciones 1
1.2. Contexto y motivación 2
1.3. Estado del arte 3
1.4. Objetivos del trabajo 4

2. Introducción específica 7
2.1. Frameworks y bibliotecas de IA/ML 7
2.2. Datasets y datos de entrenamiento 8
2.3. Infraestructura de cómputo 10

3. Diseño e implementación 13
3.1. Análisis del software 13

4. Ensayos y resultados 15
4.1. Pruebas funcionales del hardware 15

5. Conclusiones 17
5.1. Conclusiones generales 17
5.2. Próximos pasos 17

Bibliografía 19

Índice de figuras

1.1. Diagrama de bloques del sistema de AMC. 1

2.1. Ejemplo de cada modulación digital del dataset a 18 dB de SNR en el plano IQ. 9
2.2. Ejemplo de la modulación QPSK a distintos niveles de SNR en el plano IQ. 10

Índice de tablas

1.1. Enfoques de AMC y resultados reportados en la literatura. 4

2.1. Características de RadioML 2016.10A y del subconjunto utilizado. . 9
2.2. Entornos de ejecución utilizados. 10

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción general

En este capítulo se presenta el contexto general del trabajo, se describe el problema del reconocimiento automático de modulaciones y su importancia en comunicaciones digitales, se analiza el estado del arte en el área y se enuncian los objetivos que guiaron el desarrollo del trabajo.

1.1. Reconocimiento automático de modulaciones

El reconocimiento automático de modulaciones (AMC, del inglés *Automatic Modulation Classification*) es un problema fundamental en comunicaciones digitales que consiste en identificar el esquema de modulación de una señal recibida sin información previa sobre el emisor ni el canal de transmisión [1]. Entre sus aplicaciones se destacan el monitoreo del espectro radioeléctrico, la detección de emisores no autorizados, la radio cognitiva y las comunicaciones en entornos adversos [2].

Un sistema de AMC recibe como entrada muestras de una señal de radio digital en formato IQ, es decir, sus componentes en fase (I) y cuadratura (Q), y produce como salida la etiqueta del esquema de modulación identificado. El procesamiento se divide en cuatro etapas secuenciales: adquisición de la señal, preprocesamiento, clasificación mediante un modelo de aprendizaje automático y asignación de la etiqueta de modulación. Esta arquitectura de procesamiento permite configurar al receptor para demodular la señal según la modulación identificada.

En la figura 1.1 se puede observar un diagrama de bloques del sistema de AMC.



FIGURA 1.1. Diagrama de bloques del sistema de AMC.

La señal recibida se modela en banda base como $r(t) = s(t)\,c + n(t)$, donde $s(t)$ es la señal transmitida, c es un término de pérdida de trayecto o ganancia constante y $n(t)$ es ruido blanco gaussiano aditivo [3]. Esta expresión simplificada se utiliza ampliamente para desarrollar características y estadísticos de decisión, pero en los sistemas reales intervienen además efectos difíciles de modelar, como la deriva aleatoria del oscilador local, el remuestreo por el reloj del receptor y la convolución con una respuesta al impulso del canal variable en el tiempo [3]. Cada muestra de $r(t)$ es un número complejo, que se representa mediante sus componentes I y Q o, de forma equivalente, mediante la amplitud A y la fase ϕ instantáneas, con $I = A\cos\phi$ y $Q = A\sin\phi$ [4].

La relación señal a ruido (SNR, del inglés *Signal-to-Noise Ratio*) es la variable que más condiciona el desempeño de los sistemas de AMC [1, 5]. A niveles de SNR elevados (por encima de 0 dB), las distintas modulaciones presentan características diferenciables que permiten una clasificación precisa. A niveles bajos de SNR (por debajo de -10 dB), la señal es prácticamente indistinguible del ruido y la tarea de clasificación se vuelve muy difícil para cualquier método conocido [6].

1.2. Contexto y motivación

Los métodos tradicionales de AMC se basan en el análisis manual de características de la señal, tales como la transformada de Fourier, la autocorrelación, estadísticas de la envolvente y momentos de orden superior [1]. Estos métodos fueron diseñados para subconjuntos específicos de modulaciones y su desempeño se degrada significativamente en condiciones de baja SNR. Además, requieren un diseño ad hoc para cada nuevo esquema de modulación, lo que limita su escalabilidad [2].

Los métodos clásicos de AMC se agrupan en dos categorías: los basados en verosimilitud, que formulan la clasificación como un problema de decisión sobre la función de verosimilitud de la señal recibida, y los basados en características, que extraen atributos de la señal para clasificarla [2]. Estos últimos utilizan, por ejemplo, características de momentos cíclicos combinadas con clasificadores como árboles de decisión, vecinos más cercanos o máquinas de vectores de soporte [3]. Su desempeño requiere valores de SNR elevados: un algoritmo clásico de decisión alcanzó una tasa de éxito global superior al 94 % a 15 dB de SNR [7]. A bajo SNR, las redes convolucionales superaron a los clasificadores basados en características por entre 2,5 y 5 dB de SNR, mientras que por encima de 5 dB el desempeño fue similar [3]. A cambio, los enfoques de aprendizaje profundo aprenden representaciones directamente de las muestras IQ sin extracción manual de características, pero se les critica habitualmente su demanda de recursos computacionales [3].

Dentro de los métodos clásicos, los de decisión teórica plantean la clasificación como un problema de prueba de hipótesis múltiples y son óptimos en el sentido de minimizar la probabilidad de clasificación errónea. Sin embargo, sus implementaciones prácticas son costosas computacionalmente, porque suelen requerir almacenar una gran cantidad de muestras, y no son robustos ante condiciones de canal desconocidas ni ante discrepancias del receptor como el desplazamiento de la frecuencia del reloj [4]. En los métodos basados en características, la selección manual de estas es una tarea tediosa que dificulta modelar todas las discrepancias del canal [4].

El enfoque basado en aprendizaje profundo superó a los métodos clásicos y se convirtió en el paradigma dominante para AMC [8]. En particular, las redes neuronales convolucionales (CNN, del inglés *Convolutional Neural Network*) que operan directamente sobre las muestras IQ crudas de la señal lograron superar el desempeño de los métodos basados en características manuales [3]. Esta ventaja se hace especialmente evidente en condiciones de SNR moderado a alto.

Sin embargo, la búsqueda de mayor exactitud llevó al desarrollo de arquitecturas de creciente complejidad, con modelos cuya cantidad de parámetros hace inviable su implementación dentro de un procesador que pueda utilizarse en aplicaciones de radiofrecuencia en tiempo real. Esto genera una tensión fundamental:

los modelos más precisos son computacionalmente inviables para su implementación en hardware de tiempo real, que es precisamente el entorno donde AMC tiene sus aplicaciones más directas.

La propuesta de valor de este trabajo reside en buscar la mejor relación entre complejidad del modelo y exactitud de clasificación. Se investigaron diferentes arquitecturas de aprendizaje profundo con el objetivo de identificar cuál ofrece el mejor equilibrio entre desempeño y cantidad de parámetros, de modo que el modelo resultante pueda ser llevado a sistemas con recursos computacionales limitados, como puede ser un sistema embebido o una radio definida por software (SDR, del inglés *Software Defined Radio*) [9].

El presente trabajo es de carácter personal y académico y no involucra acuerdos de confidencialidad ni financiamiento externo.

1.3. Estado del arte

El estado del arte actual en AMC está dominado por métodos de aprendizaje profundo que operan directamente sobre las muestras IQ de la señal, sin extracción manual de características. Este enfoque supera sistemáticamente a los métodos clásicos basados en estadísticas de la señal, especialmente en condiciones de SNR moderado a alto.

Entre las arquitecturas de redes neuronales más utilizadas para AMC se encuentran las redes convolucionales, las redes recurrentes tipo LSTM (del inglés *Long Short-Term Memory*) y GRU (del inglés *Gated Recurrent Unit*), y las redes convolucionales temporales (TCN, del inglés *Temporal Convolutional Network*). Cada arquitectura explota diferentes propiedades de la señal: las CNN capturan patrones locales en las muestras IQ, las redes recurrentes modelan dependencias temporales a largo plazo, y las TCN combinan la capacidad de procesamiento paralelo de las CNN con un campo receptivo extendido mediante convoluciones dilatadas [3, 10, 11].

Los modelos actuales logran una exactitud promedio superior al 80 % en condiciones de SNR de 0 dB en adelante cuando se evalúan sobre el dataset RadioML 2016.10a, que es el conjunto de datos de referencia más utilizado en la literatura [3, 6]. Sin embargo, el desempeño cae drásticamente por debajo de ese umbral de SNR, lo que indica que el problema de clasificación a bajo SNR permanece abierto [6].

Entre los trabajos recientes, se propuso transformar las señales en diagramas de ojo, separar sus componentes I y Q y clasificarlas con redes convolucionales (ResNet-18, ResNet-50 y MobileNetV2) sobre un conjunto de datos sintético de 14 modulaciones con SNR de -20 a 30 dB [12]. Los autores reportaron una exactitud de 93,6 % a -20 dB de SNR, superior a la informada para modelos previos basados en redes de creencia profunda, redes recurrentes, LSTM y CLDNN, aunque con un costo computacional elevado [12].

Para las redes recurrentes, se propuso un modelo LSTM de dos capas que recibe la amplitud y la fase de la señal en el dominio del tiempo, sin características diseñadas por expertos, y que alcanzó una exactitud promedio cercana al 90 % para SNR de 0 a 20 dB sobre RadioML 2016.10a [4]. Los autores informaron que alimentar el LSTM con muestras IQ directamente dejó la exactitud constante en

9 %, mientras que la amplitud y la fase normalizadas dieron buenos resultados, y que alimentar la CNN con amplitud y fase no mejoró su exactitud respecto de la CNN con muestras IQ [4].

Las redes convolucionales temporales se aplicaron a AMC sobre muestras IQ crudas, con convoluciones causales dilatadas cuyo factor de dilatación crece con la profundidad y que amplían el campo receptivo sin requerir muchas capas [13]. Sobre un subconjunto de 8 modulaciones digitales, con vectores de 128 muestras la TCN tuvo un desempeño similar al de una red CLDNN, con una leve ventaja a SNR altos. Con vectores de 1024 muestras, regenerados a partir de RadioML, la TCN superó a la CLDNN para SNR superiores a -18 dB y alcanzó una exactitud del 100 % para SNR mayores a 0 dB [13].

En la tabla 1.1 se resumen los enfoques descritos, con la representación de entrada que utilizaron y el resultado que cada fuente reportó. Los valores no son directamente comparables entre sí, porque los conjuntos de datos y las condiciones de evaluación de cada trabajo son distintos.

TABLA 1.1. Enfoques de AMC y resultados reportados en la literatura.

Enfoque	Entrada	Resultado reportado	Fuente
Árbol de decisión, vecinos más cercanos y SVM	Características de momentos cíclicos	Superados por la CNN por entre 2,5 y 5 dB de SNR	[3]
CNN	Muestras IQ	Ventaja sobre las características de expertos a bajo SNR, sobre un dataset de 11 modulaciones	[3]
LSTM de dos capas	Amplitud y fase	Exactitud promedio cercana al 90 % para SNR de 0 a 20 dB, sobre RadioML 2016.10a	[4]
TCN	Muestras IQ	Exactitud del 100 % para SNR mayores a 0 dB, con 8 modulaciones y 1024 muestras por vector	[13]
CNN (ResNet-18, ResNet-50 y MobileNetV2)	Diagramas de ojo	Exactitud del 93,6 % a -20 dB, sobre un dataset de 14 modulaciones	[12]

1.4. Objetivos del trabajo

El objetivo general del trabajo consiste en desarrollar un sistema de clasificación automática de modulaciones de señales de radio que ofrezca la mejor relación entre exactitud de clasificación y complejidad algorítmica, con el fin de identificar modelos viables para su implementación en sistemas con recursos computacionales limitados.

Para alcanzar este objetivo se definieron los siguientes objetivos específicos:

- Analizar el dataset RadioML 2016.10a mediante la visualización de señales en el plano IQ por clase de modulación, el análisis espectral de las distintas

modulaciones y el estudio de la separabilidad de clases en función del nivel de SNR.

- Preprocesar las muestras IQ mediante normalización y dividir el dataset en conjuntos de entrenamiento, validación y prueba, manteniendo la distribución de clases y niveles de SNR.
- Investigar, diseñar e implementar arquitecturas de redes neuronales aplicables al problema de AMC, incluyendo CNN, LSTM, GRU y TCN, evaluadas tanto sobre las representaciones IQ crudas como sobre la representación en amplitud y fase.
- Entrenar los modelos y ajustar sus hiperparámetros, analizando el impacto del aumento de datos y del uso de scheduler de tasa de aprendizaje.
- Evaluar el desempeño de los modelos mediante métricas de exactitud global, exactitud en función del nivel de SNR, matriz de confusión y cantidad de parámetros del modelo.
- Comparar las arquitecturas evaluadas considerando el compromiso entre exactitud y complejidad algorítmica.
- Analizar las limitaciones del sistema desarrollado, incluyendo su comportamiento a bajo SNR y las restricciones derivadas del carácter sintético del dataset.

El alcance del trabajo no incluye la implementación del modelo en hardware de tiempo real (FPGA, microcontrolador u otros dispositivos embebidos), la prueba con señales reales capturadas mediante un receptor SDR, la medición de tiempos de inferencia en hardware específico ni el despliegue del modelo en un entorno de producción. El análisis de complejidad se limita a la comparación del número de parámetros de cada arquitectura.

Capítulo 2

Introducción específica

En este capítulo se describen las herramientas de terceros sobre las que se apoyó el trabajo: el lenguaje, los frameworks y las bibliotecas de software, el dataset RadioML 2016.10A junto con el subconjunto que se utilizó, y la infraestructura de cómputo empleada para los entrenamientos y las evaluaciones.

2.1. Frameworks y bibliotecas de IA/ML

El trabajo se desarrolló en Python, un lenguaje de código abierto administrado por la Python Software Foundation [14]. El código compartido entre los modelos y los barridos de experimentos se implementaron en módulos y scripts de Python, mientras que cada arquitectura quedó documentada en un cuaderno Jupyter, un formato de documento que combina código, resultados y explicaciones en un mismo archivo [15].

El framework de aprendizaje profundo utilizado fue PyTorch. En esta biblioteca los modelos se definen como programas regulares de Python y las derivadas necesarias para el entrenamiento se calculan mediante diferenciación automática en modo inverso, con soporte para aceleradores de hardware como las GPU (del inglés, *Graphics Processing Unit*) [16]. Con PyTorch se definieron, entrenaron y evaluaron todas las arquitecturas comparadas en el trabajo.

La elección de PyTorch se debió a tres motivos. En primer lugar, la biblioteca cuenta con numerosas herramientas para redes neuronales: capas convolucionales y recurrentes (convolución unidimensional, LSTM y GRU), normalización por lotes, regularización por dropout, la entropía cruzada como función de pérdida, optimizadores como Adam, el programador de tasa de aprendizaje ReduceLROnPlateau y cargadores de datos [17]. Además, las capas se expresan como clases de Python, lo que permite definir capas propias con libertad [16], necesidad que surgió al implementar modelos con bloques no estándar. En segundo lugar, cada uno de estos módulos cuenta con documentación oficial detallada [17]. Por último, fue el framework empleado en gran parte de la especialización, por lo que su manejo previo permitió reducir el tiempo de desarrollo.

También se utilizo biblioteca scikit-learn que reúne algoritmos de aprendizaje automático supervisado y no supervisado, e incluye iteradores que generan los índices de entrenamiento y prueba de una partición de los datos, entre ellos la validación cruzada estratificada [18]. En el trabajo se utilizó para dividir el dataset en conjuntos de entrenamiento, validación y prueba, para codificar las etiquetas de las clases y para calcular las matrices de confusión. También se utilizó para los modelos clásicos, basados en máquinas de vectores de soporte (SVM, del inglés

Support Vector Machine) con los clasificadores SVC y LinearSVC [19]. Las características de entrada de estos modelos se estandarizaron con StandardScaler, que lleva cada característica a media cero y varianza unitaria [19]; los pasos de procesamiento se encadenaron con Pipeline [18, 19] y los hiperparámetros se seleccionaron con GridSearchCV, que evalúa todas las combinaciones de una grilla de valores [18, 19].

Para el manejo de arreglos numéricos se utilizó NumPy y para el procesamiento de señales se utilizó SciPy. La visualización de los resultados se realizó con Matplotlib y con seaborn, una biblioteca de visualización estadística de datos [20].

La metadata del dataset, almacenada en formato parquet, se leyó con polars, una biblioteca de tablas de datos (*DataFrame*) cuyo núcleo está escrito en Rust [21].

Por último, se empleó la biblioteca rfml, desarrollada por Flowers y Headley [22]. Se trata de una biblioteca de aprendizaje automático para radiofrecuencia sobre PyTorch, distribuida con licencia BSD-3, que incluye conversores para cargar los datasets abiertos de DeepSig [22]. En el trabajo se utilizó para cargar los datos y para obtener el modelo de línea base. El resto del código fue desarrollado por el autor y se describe en el capítulo 3.

2.2. Datasets y datos de entrenamiento

El dataset utilizado fue RadioML 2016.10A, publicado por DeepSig. Se trata de un conjunto de datos sintético, generado con GNU Radio, que contiene 11 modulaciones, 8 digitales y 3 analógicas, a distintos niveles de SNR [23]. Es una versión más limpia y normalizada de RadioML 2016.04C, al que reemplaza [23]. El procedimiento de generación del dataset original se describe en O'Shea et al. [3], y se resume a continuación.

Según se describe en dicho trabajo, se modularon conjuntos de datos reales de voz y texto y, en el caso de las modulaciones digitales, los datos se aleatorizaron por bloques para que los bits fueran equiprobables. Cada señal atravesó un modelo de canal que incluye desvanecimiento multicamino variable en el tiempo, deriva aleatoria de los osciladores de frecuencia portadora y de reloj de muestreo, y ruido gaussiano blanco aditivo, implementado con los bloques de canal de GNU Radio. Finalmente, la señal se segmentó en ventanas rectangulares cortas [3]. Los efectos del canal producen desplazamientos temporales, cambios de escala, rotaciones y giros de la señal recibida [3].

Las modulaciones digitales del dataset son 8PSK, BPSK, CPFSK, GFSK, PAM4, QAM16, QAM64 y QPSK. El dataset contiene 1.000 ejemplos por cada combinación de modulación y nivel de SNR, con 20 niveles entre -20 y 18 dB en pasos de 2 dB, lo que totaliza 220.000 ejemplos. Cada ejemplo está formado por 128 muestras complejas, almacenadas como dos canales, I y Q. En el trabajo se utilizó únicamente el subconjunto de las 8 modulaciones digitales, con 160.000 ejemplos. En la tabla 2.1 se resumen las características del dataset y del subconjunto utilizado. Las cantidades de ejemplos y la forma de cada uno se obtuvieron de los archivos del dataset empleados en el trabajo.

TABLA 2.1. Características de RadioML 2016.10A y del subconjunto utilizado.

Característica	**Valor**
Editor	DeepSig Inc.
Generación	Sintética, con GNU Radio
Licencia	CC BY-NC-SA 4.0
Modulaciones del dataset	11 (8 digitales y 3 analógicas)
Modulaciones utilizadas	8 digitales
Niveles de SNR	20, de -20 a 18 dB en pasos de 2 dB
Ejemplos por modulación y nivel de SNR	1.000
Ejemplos del dataset completo	220.000
Ejemplos del subconjunto utilizado	160.000
Ejemplos por modulación (subconjunto)	20.000
Ejemplos por nivel de SNR (subconjunto)	8.000
Forma de cada ejemplo	2 canales (I y Q) de 128 muestras

La distribución del subconjunto es uniforme, con la misma cantidad de ejemplos para cada modulación y cada nivel de SNR. En la figura 2.1[1] se presenta un ejemplo de cada modulación digital a 18 dB de SNR en el plano IQ, con las amplitudes normalizadas por el valor máximo de cada ejemplo. Los puntos no forman constelaciones nítidas porque el canal introduce rotaciones, giros y desplazamientos de la señal [3].

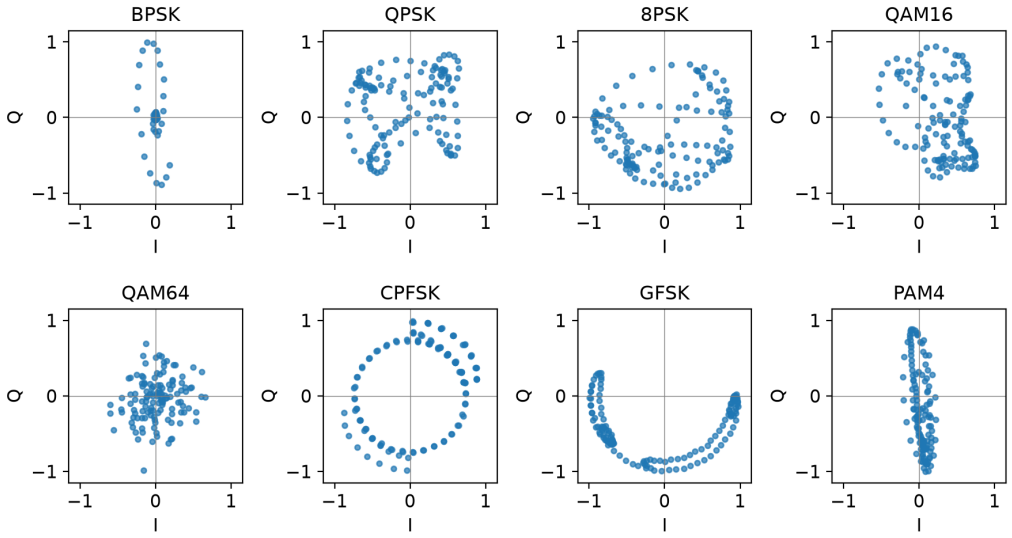


FIGURA 2.1. Ejemplo de cada modulación digital del dataset a 18 dB de SNR en el plano IQ.

En la figura 2.2[2] se muestra la misma modulación, QPSK, a distintos niveles de SNR. A 0 dB ya no se distingue la estructura observada a 18 dB, y a -10 y -20 dB los puntos forman una nube sin estructura visible.

[1]Elaboración propia a partir de los ejemplos del dataset RadioML 2016.10A de DeepSig.

[2]Elaboración propia a partir de los ejemplos del dataset RadioML 2016.10A de DeepSig.

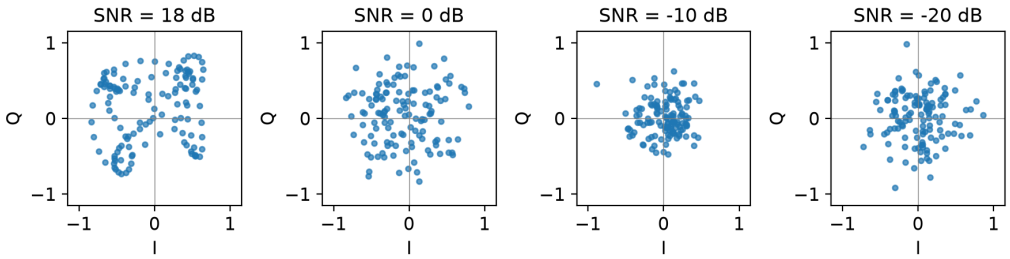


FIGURA 2.2. Ejemplo de la modulación QPSK a distintos niveles de SNR en el plano IQ.

El dataset se distribuye bajo la licencia Creative Commons Atribución, No Comercial, Compartir Igual 4.0 (CC BY-NC-SA 4.0) [23]. Esta licencia restringe su uso a fines no comerciales, condición que el presente trabajo cumple por su carácter académico.

DeepSig advierte que este dataset, producto de una investigación académica de 2016, tiene errores conocidos y no se utiliza en sus productos [23]. Además, los autores del dataset original señalan que el entrenamiento con datos reales es importante y valioso, y lo plantean como trabajo futuro [3]. Por este motivo, el desempeño sobre señales reales capturadas con un receptor queda fuera del alcance del trabajo.

Además del dataset de terceros, en parte de los experimentos se utilizaron señales sintéticas generadas con un programa desarrollado por el autor, con el fin de dar mayor robustez al entrenamiento. Este programa se describe en el capítulo 3.

2.3. Infraestructura de cómputo

Los entrenamientos y las evaluaciones se ejecutaron en una computadora personal con sistema operativo Windows 11, procesador AMD Ryzen 5 7600 de 6 núcleos y 12 hilos, 15,7 GB de memoria RAM y una GPU AMD Radeon RX 6700. Estos valores se obtuvieron del propio sistema operativo.

Para aprovechar la GPU se utilizó DirectML a través del paquete torch-directml, que permite entrenar e inferir con PyTorch en GPU compatibles con DirectX 12 [24]. Microsoft indica que esta solución se encuentra en vista previa pública [24], y AMD documenta su instalación para tarjetas Radeon RX en Windows mediante un entorno virtual de Python [25]. La última versión publicada del paquete es la 0.2.5.dev240914, de septiembre de 2024, y sus instaladores abarcan hasta Python 3.12 [26]. Por este motivo se creó un entorno virtual aparte, con Python 3.12, para las ejecuciones en GPU, mientras que el entorno principal utiliza una versión más reciente de Python y ejecuta en CPU. En la tabla 2.2 se resumen ambos entornos.

TABLA 2.2. Entornos de ejecución utilizados.

Entorno	Python	PyTorch	Dispositivo
Principal	3.13.14	2.12.1 (CPU)	CPU
DirectML	3.12.10	2.4.1 (CPU) y torch-directml	GPU AMD Radeon RX 6700

Las capas LSTM y GRU de PyTorch produjeron errores de ejecución sobre DirectML, por lo que los modelos recurrentes se entrenaron siempre en CPU. El resto de las arquitecturas se pudo ejecutar en ambos dispositivos.

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno lstlisting con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
 las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código 3.1:

```
#define MAX_SENSOR_NUMBER 3
#define MAX_ALARM_NUMBER  6
#define MAX_ACTUATOR_NUMBER 6

uint32_t sensorValue[MAX_SENSOR_NUMBER];
FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
state_t alarmState[MAX_ALARM_NUMBER];           //ON or OFF
state_t actuatorState[MAX_ACTUATOR_NUMBER];     //ON or OFF

void vControl() {

  initGlobalVariables();

  period = 500 ms;

  while(1) {

    ticks = xTaskGetTickCount();

    updateSensors();

    updateAlarms();

    controlActuators();

    vTaskDelayUntil(&ticks, period);
  }
```

```
28 }
```

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se puedo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

[1] Z. Zhu y A. K. Nandi. *Automatic Modulation Classification: Principles, Algorithms and Applications*. Wiley-IEEE Press, 2015. ISBN: 978-1-118-87316-0.

[2] O. A. Dobre et al. «Survey of automatic modulation classification techniques: classical approaches and new trends». En: *IET Communications* 1.2 (2007), págs. 137-156. DOI: 10.1049/iet-com:20050176.

[3] T. J. O'Shea, J. Corgan y T. C. Clancy. «Convolutional radio modulation recognition networks». En: *Proc. Engineering Applications of Neural Networks (EANN)*. arXiv:1602.04105. 2016.

[4] S. Rajendran et al. *Deep Learning Models for Wireless Signal Classification with Distributed Low-Cost Spectrum Sensors*. arXiv:1707.08908, v2. 2018. DOI: 10.48550/arXiv.1707.08908.

[5] J. G. Proakis y M. Salehi. *Digital Communications*. 5.ª ed. McGraw-Hill, 2008. ISBN: 978-0-07-295716-7.

[6] T. J. O'Shea, T. Roy y T. C. Clancy. «Over-the-air deep learning based radio signal classification». En: *IEEE Journal of Selected Topics in Signal Processing* 12.1 (2018), págs. 168-179. DOI: 10.1109/JSTSP.2018.2797022.

[7] A. K. Nandi y E. E. Azzouz. «Algorithms for automatic modulation recognition of communication signals». En: *IEEE Transactions on Communications* 46.4 (1998), págs. 431-436.

[8] T. J. O'Shea y J. Hoydis. «An introduction to deep learning for the physical layer». En: *IEEE Transactions on Cognitive Communications and Networking* 3.4 (2017), págs. 563-575. DOI: 10.1109/TCCN.2017.2758370.

[9] J. Mitola. «The software radio architecture». En: *IEEE Communications Magazine* 33.5 (1995), págs. 26-38. DOI: 10.1109/35.393001.

[10] S. Peng et al. «Modulation classification based on signal constellation diagrams and deep learning». En: *IEEE Transactions on Neural Networks and Learning Systems* 30.3 (2019), págs. 718-727. DOI: 10.1109/TNNLS.2018.2850703.

[11] B. Kim et al. «Automatic modulation classification using recurrent neural networks». En: *IEEE Communications Letters* (2018). DOI: 10.1109/LCOMM.2018.2792026.

[12] F. U. Rehman, Q. Abbas y M. K. Shehzad. *DL-AMC: Deep Learning for Automatic Modulation Classification*. arXiv:2504.08011. 2025. DOI: 10.48550/arXiv.2504.08011.

[13] A. Ghasemi, C. Parekh y P. Guinand. *Spectrum Awareness Under Co-Channel Usage Via Deep Temporal Convolutional Networks*. Communications Research Centre, Ottawa. https://winnforum.org/assets/Proceedings/2019/TS3.3%20Ghasemi%20paper.pdf. 2019.

[14] Python Software Foundation. *About Python*. https://www.python.org/about/. Acceso: 9 de noviembre de 2026.

[15] T. Kluyver et al. «Jupyter Notebooks - a publishing format for reproducible computational workflows». En: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. Año a confirmar en la fuente. IOS Press, 2016, págs. 87-90. DOI: 10.3233/978-1-61499-649-1-87.

[16] A. Paszke et al. «PyTorch: An imperative style, high-performance deep learning library». En: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. arXiv:1912.01703. 2019.

[17] PyTorch Contributors. *PyTorch documentation (versión 2.15)*. https://docs.pytorch.org/docs/2.15/index.html. Acceso: 9 de noviembre de 2026. Páginas consultadas: nn, optim y data.

[18] F. Pedregosa et al. «Scikit-learn: Machine learning in Python». En: *Journal of Machine Learning Research* 12.85 (2011), págs. 2825-2830.

[19] scikit-learn developers. *scikit-learn documentation (versión 1.9.1)*. https://scikit-learn.org/stable/. Acceso: 9 de noviembre de 2026. Páginas consultadas: SVM, preprocesamiento, Pipeline y GridSearchCV.

[20] M. L. Waskom. «seaborn: statistical data visualization». En: *Journal of Open Source Software* 6.60 (2021), pág. 3021. DOI: 10.21105/joss.03021.

[21] Polars. *Polars: documentación oficial*. https://docs.pola.rs/. Acceso: 9 de noviembre de 2026.

[22] B. Flowers y W. C. Headley. *Radio Frequency Machine Learning (RFML) in PyTorch*. Repositorio de GitHub, https://github.com/brysef/rfml. Acceso: 9 de noviembre de 2026. Licencia BSD-3-Clause.

[23] DeepSig Inc. *Datasets: RadioML 2016.10A*. https://www.deepsig.ai/datasets. Acceso: 9 de noviembre de 2026.

[24] Microsoft. *Enable PyTorch with DirectML on Windows*. https://learn.microsoft.com/en-us/windows/ai/directml/gpu-pytorch-windows. Acceso: 9 de noviembre de 2026.

[25] AMD Research. *Setting PyTorch environment for GPU (Windows)*. AUP AI Tutorials, https://amdresearch.github.io/aup-ai-tutorials/env/env-gpu-windows.html. Acceso: 9 de noviembre de 2026. 2025.

[26] Microsoft. *torch-directml 0.2.5.dev240914*. Python Package Index, https://pypi.org/project/torch-directml/. Acceso: 9 de noviembre de 2026.